

1 ☐ WEEK 01 COACHING SESSION TRANSCRIPT

2 ☐ Learning Coach and Software Mentor Session

3 ☐ =====

4 ☐

5 ☐ COACH: Welcome! Just to be clear from the start, this is a coaching session, not a test. It is perfectly okay
–and even encouraged—to say "I don't know" or "I made a mistake." That's the fastest way we'll learn today.

6 ☐

7 ☐ This is a safe learning space, Daniel – mistakes are how we find the gaps worth closing. Since we're chatting
here, whenever we get into code, just paste the relevant snippet in and we'll dig into it together.

8 ☐

9 ☐ We've got four topics to cover today: Express routing, JavaScript language principles, EJS partials/pages,
and frontend CSS – but you can lead with whatever feels most natural to start.

10 ☐

11 ☐ So, to kick things off: which part of your server.js are you proudest of, or found trickiest to get working –
and could you paste that routing snippet in?

12

13 ☐ ---

14

15 ☐ STUDENT:

16 ```javascript

17 ☐ // console.log("Hello, Node.js!");

18 import express from 'express';

19 ☐

20 ☐ import { fileURLToPath } from 'url';

21 ☐

22 import path from 'path';

23 ☐

24 ☐ const __filename = fileURLToPath(import.meta.url);

25 const __dirname = path.dirname(__filename);

26

27 const NODE_ENV = process.env.NODE_ENV?.toLowerCase() || 'production';

28 const PORT = process.env.PORT || 3000;

29

30 const app = express();

31

32 /**

33 * Configure Express middleware

34 ☐

35 */

36 // Serve static files from the public directory

37

38 ☐ app.use(express.static(path.join(__dirname, 'public')));

39 ☐

40 ☐ // Set EJS as the templating engine

41 ☐ app.set('view engine', 'ejs');

```
42 ☐
43 ☐ // Tell Express where to find your templates
44 ☐ app.set('views', path.join(__dirname, 'src/views'));
45 ☐
46 ☐ /**
47 ☐  * Routes
48 ☐  */
49 ☐ app.get('/', (req, res) => {
50 ☐     const title = 'Home';
51 ☐     res.render('home', { title });
52 ☐ });
53 ☐
54 app.get('/organizations', (req, res) => {
55 ☐     const title = 'Our Partner Organizations';
56     res.render('organizations', { title });
57 });
58
59 app.get('/projects', (req, res) => {
60     const title = 'Service Projects';
61 ☐     res.render('projects', { title });
62 });
63 ☐
64 ☐ app.get('/categories', (req, res) => {
65 ☐     const title = "Categories";
66     res.render("categories", { title })
67 })
68
69 app.listen(PORT, () => {
70 ☐     console.log(`Server is running at http://127.0.0.1:${PORT}`);
71     console.log(`Environment: ${NODE_ENV}`);
72 });
73 ```
74 ☐
75 ☐ Hello, what I'm most proud of my code is that I was able to understand how to make a route and use the .get
    method to render the webpage of that route, also I was able to understand about res and req. So for example,
    in the categories route, the request would be when I do something that /categories path and the response
    would be the output, for example render the website "categories". In other hand, what I found tricky to
    understand was the __filename and __dirname, and its declarations, also the listen function and the
    declarations of NODE_ENV and PORT
76 ☐
77 ☐ ---
78 ☐
79 ☐ COACH: Nice, thanks for sharing that – solid, clean routing setup.
80 ☐
```

81 ☐ Let's start with what you said felt trickiest: the `__filename`/`__dirname`` lines. Looking at those two lines together – `fileURLToPath(import.meta.url)`` and `path.dirname(__filename)`` – what do you think each one is actually producing, and why do you think you need both instead of just one?

82 ☐

83 ---

84 ☐

85 ☐ STUDENT: Well, I believe the `fileURLToPath(import.meta.url)` is importing the url, for e.g the local url so that I can update it later on using the `server.js` file. The `path.dirname(__filename)` is importing actual directory where I'm at based on the root URL.

86 ☐

87 ☐ ---

88 ☐

89 ☐ COACH: That's a reasonable guess, but let's untangle it a bit – you're close on the second part, off on the first. Let's isolate just the first line.

90

91 `import.meta.url`` isn't importing anything from outside – it's a built-in value that JS gives you automatically inside an ES module. What do you think that value actually *\*contains\** – i.e., if you did `console.log(import.meta.url)`` right at the top of your file, what kind of string do you think would print out?

92

93 ---

94

95 STUDENT: I don't know

96

97 ---

98 ☐

99 ☐ COACH: Totally fine – "I don't know" is exactly the right thing to say here, and it's an easy one to test rather than guess.

100 ☐

101 Here's a hint: `import.meta.url`` gives you the *\*file's own location\**, expressed as a `file://`` URL (something like `file:///Users/daniel/project/server.js``) – not a website URL, not something you'd visit in a browser.

102

103 Given that, why do you think `fileURLToPath()`` needs to be called on it before it's useful for something like `path.join(__dirname, 'public')``?

104

105 ---

106

107 ☐ STUDENT: Because it is going to run in the server and it must have an URL path instead of a local path. Since we don't people don't have the same computer to test it in the local mode with the same files organizations, they may save a file inside a completely different directory, for example. I can save it in documents and they instead save it in downloads

108

109 ---

110

111 COACH: Let's slow down on this one – I think the direction is flipped from what's actually happening.

112

113 ☐ `fileURLToPath()` doesn't turn a path into a URL – it does the opposite: it takes that `file:///...` URL string and converts it into a normal OS file path (like `/Users/daniel/project/server.js` or `C:\Users\daniel\project\server.js`), which is what functions like `path.dirname()` and `path.join()` expect to work with.

114

115 ☐ So here's the real question behind why this dance is even necessary: in old-style Node.js (CommonJS, using `require`), `__dirname` was just automatically available – no setup needed. But you're using `import express` from `'express'` – ES modules. Why do you think ES modules *don't* give you `__dirname` for free, forcing you to build it yourself from `import.meta.url`?

116

117 ☐ ---

118

119 STUDENT: I don't know

120

121 ---

122

123 COACH: No problem – this is genuinely a "just know the fact" kind of gap, not a reasoning failure on your part.

124

125 The short answer: `__dirname` and `__filename` were a **CommonJS-only feature** – Node basically injected those variables automatically into every CommonJS file behind the scenes. When Node added support for ES modules (the `import`/`export` syntax you're using), it deliberately did **not** carry that over, partly because ES modules are meant to be usable in browsers too, where there's no such thing as a file path. So `import.meta.url` is the one thing ES modules *do* give you – and then it's on you to convert it into the old-style `__dirname` yourself if you need it. That's exactly what those two lines do, and it's such a common need that it's basically boilerplate you'll see at the top of almost every ES module Node server.

126

127 Good – that clears up the trickiest part. Now let's shift into your routes themselves, since you said those felt solid.

128

129 Looking at your four `app.get()` routes – they all follow basically the same pattern (define a title, call `res.render()`). If you had to add, say, 20 more pages like this, what do you think might become a problem with just copy-pasting this same pattern over and over, and is that something Express gives you tools to avoid?

130

131 ---

132

133 STUDENT: i still don't get the idea of `__dirname` and `__filename`

134

135 ---

136

137 COACH: No worries – let's make it concrete instead of abstract, since the words alone don't mean much without seeing actual values.

138

139 Imagine your project lives at `/home/daniel/my-app/` and your server file is `server.js`, sitting directly inside that folder.

140

141 - `\_\_filename` – the *full path to server.js itself*, i.e. the string `/home/daniel/my-app/server.js`
142 - `\_\_dirname` – just the *folder that file lives in*, with the filename chopped off, i.e. `/home/daniel/my-app`
143
144 That's it. `\_\_filename` = "where am I, exact file." `\_\_dirname` = "what folder am I sitting in." And
`path.dirname()` is literally the function that does that chopping-off – you feed it a file path, it hands back
the folder path.
145
146 So here's my question: once you have `\_\_dirname` = `/home/daniel/my-app`, why do you think your code then does
`path.join(\_\_dirname, 'public')` instead of just writing `'public'` directly in `express.static('public')`?
147
148 ---
149
150 STUDENT: because It needs to go to the directory and find the public folder where all the static files are
151 
152 ---
153
154 COACH: That's the right instinct – it needs to *find* that folder – and here's the extra piece that makes it
click: if you just wrote `'public'` as a plain relative string, Express would look for it relative to wherever
you happened to run the `node server.js` command *from* (your current terminal location), not relative to where
`server.js` itself lives. If someone ran that command from a different folder, `'public'` alone could point to
the wrong place or nowhere at all. Using `\_\_dirname` locks it to "public folder, right next to my server file,
no matter where I launched the command from." That's the whole payoff of the boilerplate.
155
156 Good, I think that one's landed. Let's move to your actual routes now – we're still on Topic 1 (Express
routing).
157
158 Looking at your four `app.get()` calls, they all repeat almost the same shape: set a `title`, call
`res.render(page, { title })`. If you had to add 20 more pages like this, what do you think would start to feel
painful about copy-pasting this pattern each time, and do you have a guess whether Express gives you any tools
to avoid that repetition?
159
160 ---
161
162 STUDENT: No, I don't. What would it be?
163
164 ---
165
166 COACH: Fair enough – let's build it up with a hint rather than me just handing you the pattern.
167
168 Think about what's actually *varying* between your four routes versus what's staying the same. The path (`/`,
`/organizations`, `/projects`, `/categories`) changes, and the `title` changes, and which template gets
rendered changes – but the *shape* of the function is identical every time: grab a title, render a page with
that title.
169

170 One common approach is a ****data-driven route**** – instead of four separate `app.get()` calls, you'd have one array or object mapping paths to their titles/templates, and then a single loop (or a small helper function) that registers a route for each entry.

171

172 Here's my one question: given that idea, what do you think the tradeoff would be – is turning your four explicit, readable `app.get()` calls into a loop-generated version actually a **good** idea at this stage of your project, or could it make the code harder to follow for someone (including future-you) reading it?

173

174 ---

175

176 STUDENT: It would be a great Idea, because the code will be simple and shorter, It's going to be easier to maintain in the future too

177

178 ---

179

180 COACH: That's true in the abstract, but let's stress-test it against your **actual** situation, because "shorter and more maintainable" isn't automatically true here.

181

182 Right now, all four of your routes do the exact same trivial thing: render a template with a title, no other logic. But real apps rarely stay that simple – what happens when, say, `/projects` later needs to also query a database for a list of projects, or `/categories` needs to handle a form submission? If those routes are generated from a loop over a data array, how would you add unique logic to just **one** of them without it getting awkward?

183

184 Given that, would you still say the loop is clearly better for a 4-route app like this – or might there be a size/complexity threshold where it starts to pay off?

185

186 ---

187

188 STUDENT: if that's the case, then It wouldn't be a good Idea, since the routes have completely different scopes

189

190 ---

191

192 COACH: Exactly right – that's the real insight: a loop is great when routes are truly uniform, but the moment routes need to diverge (different logic, different data, different validation), forcing them into a generic loop just makes you fight the abstraction. For a 4-route site like yours right now, explicit `app.get()` calls are actually the **better** choice – clear, easy to debug, easy to extend individually. That's a useful rule of thumb to keep: don't abstract until the repetition is actually causing pain.

193

194 Alright, that wraps up Topic 1 (Express Routing) nicely. Let's move to Topic 2: JavaScript language principles.

195

196 Could you paste a JavaScript function from your project – ideally one with some actual logic in it (not just a one-liner) – that you wrote or modified yourself?

197

198 ---

```
199
200 STUDENT:
201 ```javascript
202 // Set EJS as the templating engine
203 app.set('view engine', 'ejs');
204
205 // Tell Express where to find your templates
206 app.set('views', path.join(__dirname, 'src/views'));
207 ```
208 ☐
209 ☐ ---
210 ☐
211 ☐ COACH: This is actually just configuration – two `app.set()` calls telling Express which template engine to
    use and where to find templates. There's no real logic here (no conditionals, loops, calculations, or custom
    behavior), so it's not quite what I'm looking for to dig into JavaScript language principles like scope,
    closures, arrow functions, array methods, async/await, etc.
212
213 ☐ Do you have a function anywhere in your project – maybe client-side JS in a `public/js` file, or something
    handling form logic, data transformation, or an event listener – that has some actual logic inside it, even a
    small `if` statement or a loop?
214 ☐
215 ---
216 ☐
217 ☐ STUDENT: app.listen(PORT, () => {
218   console.log(`Server is running at http://127.0.0.1:${PORT}`);
219   console.log(`Environment: ${NODE_ENV}`);
220 });
221
222 ---
223
224 COACH: Okay, this one at least has an actual function body we can dig into – the arrow function `() => {...}`
    passed to `app.listen`.
225
226 Let's start with the syntax itself: this is written as an arrow function rather than something like
    `function() {...}`. In your own words, what do you understand an arrow function to be, and why do you think it
    was used here instead of a regular `function` keyword?
227
228 ---
229
230 STUDENT: Well, the arrow function, it's basically a function that is wrote when something specific is true, for
    example if PORT == true then it will do the following. I honestly don't remember the difference between regular
    functions, I will say it's to avoid redundancy and avoid the add unnecessary scopes inside functions
231
232 ---
233
```

234 COACH: I can see there's a mix-up here – let's clear it up, because arrow functions have nothing to do with conditionals like ``if PORT == true``.

235

236 ☐ An arrow function is just **another way to write a function** – same concept as ``function() {}``, just different syntax (and one real behavioral difference around ``this``, which we can get to later). It doesn't check anything or run conditionally by itself; it's a value – a block of code – that you're *handing to* ``app.listen`` as its second argument, and ``app.listen`` is the one that decides *when* to actually call it (once the server successfully starts).

237

238 So here's a simpler question to rebuild from: in ``app.listen(PORT, () => { ... })``, when do you think that arrow function actually runs – immediately when your file loads top to bottom, or at some later point?

239

240 ☐ ---

241

242 STUDENT: I believe it runs when it loads top to bottom

243

244 ---

245 ☐

246 COACH: That's a really common assumption, but it's actually the opposite – and this is a key JS concept called a **callback function**.

247

248 Here's the thing: ``app.listen(PORT, callback)`` starts the process of binding your server to that port – but that takes a small amount of time (setting up the OS-level network socket). Your JS file doesn't pause and wait for that to finish. Instead, you're handing Express that arrow function and saying "once you're actually done setting up and the server is really running, *then* call this." So the two ``console.log`` lines inside it don't run "on the way down" through your file – they run *later*, as a reaction to the server successfully starting.

249 ☐

250 This is the same pattern you'll see all over Node: pass a function as an argument, and something else calls it back when an event happens (a request comes in, a file finishes reading, a timer expires) – hence the name "callback."

251

252 Does that distinction – "defined now, but *run later* in response to something" – make sense, or is there a part of that still fuzzy?

253

254 ---

255

256 STUDENT: It makes sense now, so it basically just outputs again the url everytime you refresh the code, so that it keeps track of it and save the changes to the url

257

258 ☐ ---

259

260 COACH: Let's correct that, because there's a mix-up about *what* triggers this callback.

261

262 ☐ This callback runs **once** – when the server first starts up (i.e., when you run ``node server.js`` and it finishes binding to the port). It does *not* run every time someone refreshes a webpage in the browser, and

it's not "saving changes to a URL." Refreshing a browser page triggers a completely different thing – a new HTTP request coming in, which would hit one of your `app.get()` route handlers instead (the ones we discussed with `/``, `/organizations``, etc.), not this `listen`` callback.

263

264 So to separate these two clearly: what do you think is different about `*when*` `app.listen``'s callback fires versus when one of your `app.get('/organizations', (req, res) => {...})`` callbacks fires?

265

266 ---

267

268 STUDENT: The `app.listen` waits for a change and the `app.get` is when you refresh the page or you redirect yourself towards another

269

270 ---

271

272 COACH: We're close but let's nail down the `*trigger*` more precisely, because "waits for a change" isn't quite it either.

273

274 `app.listen``'s callback fires from `**one single event**`: the server successfully starting up and binding to the port. That's it – it happens once, right after you run your file, and never again (unless you restart the server).

275

276 `app.get('/organizations', ...)``'s callback fires from a `**different kind of event, repeatedly**`: every single time a browser (or anything) sends an HTTP GET request to that specific path – `/organizations``. So yes, refreshing the page or navigating there triggers it, but so would someone else's browser hitting that URL, or you visiting it 50 times in one session – it fires fresh, every time, for every visit.

277

278 So the real contrast is: `**one-time "the app is ready" event**` vs. `**repeatable "someone just asked for this page" event**`. Does that distinction land now?

279

280 ---

281

282 ☐ STUDENT: Yes, it does

283 ☐

284 ☐ ---

285

286 COACH: Great, glad that clicked – that "one-time startup event vs. repeatable per-request event" distinction is genuinely core to how Node servers work, so it's worth having solid.

287

288 That wraps up a good chunk of Topic 2 already through the callback discussion. Let's dig into one more JS principle while we're here, using that same snippet.

289

290 Look at the `console.log` lines: ``` `Server is running at http://127.0.0.1:${PORT}` ```. What is that backtick syntax with ``${PORT}`` inside called, and what do you think it's doing differently compared to writing something like ``Server is running at http://127.0.0.1:' + PORT``?

291

292 ---
293
294 STUDENT: you must use placeholders like `${}` to store variables when storing or calling a backtick string. Also, when we use placeholders instead of plus sign method, it helps our code maintenance and readability
295
296 ---
297 ☐
298 ☐ COACH: That's the right idea – you've described the *behavior* well. The proper name for that backtick syntax is a *template literal* (or template string), and ``${...}`` inside it is called *string interpolation* – you're right that it "stores" or rather *evaluates and inserts* a variable's value directly into the string. And yes, your readability point is exactly why it's preferred over ``+`` concatenation, especially once you're mixing multiple variables into one string.
299
300 Good, that's a solid pass on Topic 2. Let's move to Topic 3: EJS partials and pages.
301
302 Could you paste an EJS snippet – ideally something showing either a partial (like a header/footer/nav you're including into multiple pages) or a page that uses some EJS logic (like `<% if %>` or `<% forEach %>`)?
303
304 ---
305
306 STUDENT:
307 `<%- include('partials/header') %>`
308 `<main>`
309 `<h1><%= title %></h1>`
310 `<h2>Welcome to the CSE 340 Service Network!</h2>`
311 `<p>Our mission is to promote service across the world by connecting volunteers with service opportunities in their community.</p>`
312
313 ``
314 `</main>`
315 `<%- include('partials/footer') %>`
316
317 ---
318
319 COACH: Good, this is a nice one to dig into – there's an important distinction hiding in here.
320
321 Notice you're using two *different* EJS tags: `<%- include(...) %>` on the header/footer lines, but `<%= title %>` for the heading. Both output something into the page, but they use different symbols (``-`` vs ``=``). What do you understand the difference to be between `<%-`` and `<%=``, and why do you think each one was used where it was?
322
323 ---
324
325 STUDENT: The `<%-` doesn't escape any HTML and the `<%=` escapes anything, it read it as it is
326

327 ---
328
329 COACH: That's exactly right – nicely put. ``<%=` **escapes** HTML (so if `title` somehow contained something like ``, it'd print as literal text `` on the page instead of making it bold), which protects you from things like accidental or malicious HTML/script injection. `<%-` does **not** escape – it outputs raw HTML as-is, which is exactly what you want for `include()`, since you *want* the header/footer's actual HTML tags to render as real HTML, not show up as text on the page.`

330
331 Good instinct using ``<%=` for `title` then – since `title` is just plain text, escaping is the safe default. Now let's connect this to your `partials/header` file specifically.`

332
333 What kind of content did you put inside ``partials/header.ejs` – and why did you decide to break that piece out into its own separate partial file instead of just writing it directly into every page like `home.ejs`, `organizations.ejs`, etc.?`

334
335 ---
336
337 STUDENT:
338 ```html
339 <!DOCTYPE html>
340 <html lang="en">
341 <head>
342 <meta charset="UTF-8">
343 <meta name="viewport" content="width=device-width, initial-scale=1.0">
344 <title><%= title %></title>
345 <link rel="stylesheet" href="/css/main.css">
346 </head>
347 <body>
348 <nav>
349
350 Home
351 Organizations
352 Projects
353 Categories
354
355 </nav>
356
357 ```

358
359 I decided to move in into a partial file because this is where I was going to store all the repetitive html code and outside of it will be the unique files that has unique code

360
361 ---
362

363 COACH: That's a solid grasp of the core idea – avoiding repetition of shared markup (the `<head>`, the nav, the doctype) across every page is exactly the right reason to use a partial. Nicely done.

364

365 One thing worth noticing here, though: this file is doing something slightly unusual – it opens `<html>`, `<head>`, `<body>`, and `<nav>` but never closes any of them (no `</body>`, `</html>`). Why do you think it's okay – or maybe *necessary* – for `header.ejs` to leave those tags unclosed, given how you're using `include('partials/header')` in `home.ejs`?

366

367 ---

368

369 STUDENT: because the content hasn't finished yet, if you do that, then it will insert `</html` or `</body` right before ending your code

370

371 ---

372

373 COACH: Exactly – you've got it. The header is only the *first half* of the HTML document; it deliberately leaves those tags open because the actual page content (the `<main>` block in `home.ejs`) needs to land *inside* `<body>`, not after it. Then `footer.ejs` presumably closes out `</body>` and `</html>`, completing the structure. So really, `header.ejs` + page content + `footer.ejs` together form one valid, complete HTML document; none of the three pieces is valid HTML on its own, and that's expected.

374

375 Good – that covers Topic 3 well. Let's move to our last topic: Topic 4, frontend CSS.

376

377 Could you paste a CSS snippet from your `main.css` – ideally something with more than just basic color/font settings, like a layout technique (flexbox, grid, positioning) or a responsive rule (`@media`)?

378

379 ---

380

381 STUDENT: [pasted full main.css file – CSS reset & variables, accessibility/global styles, header/nav, layout/main content, category links/sidebar nav, organizations list, projects list, footer, and mobile responsiveness via `@media (max-width: 600px)`]

382

383 ---

384

385 COACH: This is genuinely well-organized CSS – nice use of custom properties and a clear section structure.

386

387 Let's dig into one clever piece of layout hiding in here: look at `body` (`display: flex; flex-direction: column; min-height: 100vh;`), then `main` (`flex: 1;`), then `footer` (`margin-top: auto;`). Together, these three rules create a common pattern – the "sticky footer" effect, where the footer stays pinned to the bottom of the viewport even on pages with very little content, instead of floating up in the middle of the page.

388

389 In your own words, why do you think `main { flex: 1; }` is the key piece that makes this work – what do you think would happen to the layout if you removed just that one line?

390

391 ---

392 ☐

393 ☐ STUDENT: it basically sets the priority

394 ☐

395 ☐---

396 ☐

397 ☐ COACH: Let's get more precise than "priority," because that's a bit vague for what's actually happening.

398 ☐

399 ☐ Here's a hint to build from: ``flex: 1`` is shorthand that mainly sets ``flex-grow: 1``. In a flex container (which ``body`` is, since it has ``display: flex; flex-direction: column``), ``flex-grow`` controls how much a child **expands to fill leftover empty space** in the container.

400 ☐

401 ☐ So think about it this way: ``body`` is set to ``min-height: 100vh`` – meaning it's *at least* the full height of the screen. If your page content (nav + main + footer) is short, there's leftover empty space in that 100vh column. Given that ``main`` has ``flex-grow: 1`` and nav/footer don't, which element do you think ends up stretching to consume that leftover space?

402 ☐

403 ☐---

404 ☐

405 STUDENT: the main

406

407 ☐---

408

409 ☐ COACH: Exactly right – ``main`` stretches to fill all the leftover vertical space, since it's the only child with ``flex-grow: 1``. That's what pushes ``footer`` down to the very bottom of the viewport even when the page content is short, instead of the footer floating up right under a small amount of text. And that's also why ``margin-top: auto`` on the footer isn't even strictly necessary here – ``main`` growing already does the heavy lifting – but it's a nice belt-and-suspenders safeguard.

410

411 That covers all four topics nicely. Quick recap of where we landed:

412

413 ☐ - Express routing: understood `__dirname/__filename`, the callback nature of `app.listen`, and correctly reasoned that explicit routes beat premature loop-abstraction for a small app.

414 - JS principles: arrow functions as callbacks (not conditionals), template literals/string interpolation.

415 - EJS: `<%-` vs `<%=` escaping, and why `partials` intentionally contain unclosed tags.

416 - CSS: the flexbox sticky-footer pattern and what `flex-grow` actually does.

417

418 ☐=====

419 END OF SESSION TRANSCRIPT

420